

## บทที่ 6

# คอมโพเนนต์ .NET (Assembly)

### 6.1 วิวัฒนาการของคอมโพเนนต์

ในยุคสมัยที่ระบบปฏิบัติการยังใช้ดอส (DOS) กันอยู่นั้น โปรแกรมที่เขียนขึ้นเป็นเพียงโปรแกรมเล็กๆ เท่านั้น แต่ในยุคสมัยที่วินโดวส์ 16 บิตได้ออกมา การเขียนโปรแกรมบนวินโดวส์นั้นจะมีขนาดใหญ่กว่าการเขียนโปรแกรมบนดอสมาก จึงได้มีการพัฒนาแชร์ไลบรารี (Share Library) ขึ้นมา เพื่อลดขนาดของโปรแกรมที่พัฒนาบนวินโดวส์ลง ซึ่งแชร์ไลบรารีนี้ก็คือ ไดนามิกลิงค์ไลบรารี (Dynamic Linked Library - DLL) นั่นเอง

มาถึงยุคสมัยวินโดวส์ 95 วินโดวส์ 95 ได้กลายเป็นระบบปฏิบัติการหลักที่มาแทนที่ดอส ทำให้ทางผู้พัฒนาแอปพลิเคชันพัฒนาแอปพลิเคชันต่างๆ ที่ใช้กับวินโดวส์ 95 ขึ้นมามากมาย ซึ่งจากการที่มีบริษัทที่พัฒนาแอปพลิเคชันอยู่หลายบริษัท ต่างฝ่ายต่างทำการพัฒนาแอปพลิเคชันของตน ทำให้มีการใช้ชื่อ DLL ที่เหมือนกัน เมื่อผู้ใช้งานแอปพลิเคชันที่มี DLL เหมือนกัน แต่คนละบริษัทที่พัฒนา ทำให้เกิดการทับกันของ DLL ซึ่งอาจทำให้ไม่สามารถใช้งานแอปพลิเคชันแรกที่ได้ลงไว้แล้วได้เลย หรืออาจจะเกิดจากการที่บริษัทผู้พัฒนาได้มีการพัฒนา DLL เวอร์ชันใหม่ขึ้นมา แอปพลิเคชันเก่าที่ใช้ DLL เวอร์ชันเก่าอาจจะไม่สามารถใช้งาน DLL เวอร์ชันใหม่ได้

ทางไมโครซอฟท์จึงได้ทำการพัฒนา DLL ใหม่ขึ้น ซึ่งรู้จักกันในนามคอมโพเนนต์ (COM Component) ซึ่งจะมีความสามารถในการทำอินเทอร์เฟซอินเฮอริเทนต์ (Interface Inheritance) โดยใน DLL ชนิดเดียวกันจะมีหลายเวอร์ชันได้ โดยที่ DLL เวอร์ชันเก่าจะต้องเข้ากันได้กับ DLL เวอร์ชันใหม่ด้วย

จากการที่ไมโครซอฟท์ได้พัฒนา DLL รุ่นที่ 2 ขึ้นมา แต่กลับแก้ปัญหาได้ไม่ดีเท่าไรนัก ทางไมโครซอฟท์จึงได้พัฒนา DLL รุ่นที่ 3 ขึ้นมาโดยใช้เทคโนโลยีที่เรียกว่า ฟิวชันเทคโนโลยี (Fusion Technology) โดยมีแนวทางที่ว่า DLL แต่ละตัวจะถือว่าเป็นคนละแฟ้มข้อมูลกัน เพื่อไม่ให้เกิดการทับกัน ซึ่งในรุ่นที่ 3 นี้ DLL จะไม่เพียงแต่เก็บโค้ด แต่จะเก็บข้อมูลอธิบายถึงองค์ประกอบของ DLL รวมทั้งเวอร์ชันของ DLL แต่ละตัวด้วย เรียกว่าเมตาดต้า (Metadata) ซึ่ง DLL รุ่นที่ 3 นี้ ทางไมโครซอฟท์เรียกว่า แอสเซมบลี (Assembly) โดยแอสเซมบลีจะมีอยู่ 2 ชนิดด้วยกัน คือ

1. ไพรเวท แอสเซมบลี (Private Assembly) ซึ่งจะเก็บอยู่ในโฟลเดอร์ที่ติดตั้งแอปพลิเคชัน โดยจะไม่ไปยุ่งเกี่ยวกับโฟลเดอร์อื่นๆ
2. แชร์แอสเซมบลี (Shared Assembly) แอสเซมบลีชนิดนี้จะถูกเก็บอยู่ใน โกลบอลแอสเซมบลีแคช (Global Assembly Cache - GAC) ซึ่งจะอยู่ใน WINNT\Assembly\GAC โดยเพื่อให้ไม่เกิดการทับกันขึ้น ทางไมโครซอฟท์ใช้หลักการเข้ารหัสแบบ RSA โดยที่กำหนดให้บริษัททุกบริษัทที่จะนำเอาแอส

แอมบลิ ไปเก็บไว้ใน GAC ได้นั้นต้องสร้างคีย์ (Key) 2 ตัว นั่นก็คือคีย์สาธารณะ (Public Key) และคีย์ส่วนตัว (Private Key) โดยทางไมโครซอฟท์จะนำเอา 8 ไบต์หลังของคีย์สาธารณะของบริษัทนั้นมาสร้างเป็นชื่อไคเรกทอรี เช่น 1.0.2411.0\_\_b77a5c561934e089 ส่วนแรก 1.0.2411.0 คือเวอร์ชัน และส่วนที่สอง b77a5c561934e089 นั้นเป็น 8 ไบต์สุดท้ายของคีย์สาธารณะของไมโครซอฟท์ โดยชื่อไคเรกทอรีที่เกิดจากคีย์สาธารณะนี้ ทางไมโครซอฟท์ได้เรียกว่า สตรองเนม (Strong Name)

## 6.2 Assemblies

ในระหว่างยุคของ COM เอกสารของไมโครซอฟท์ ใช้คำว่า คอมโพเนนต์ ในความหมายของ COM คลาส หรือ COM โมดูล (DLLs หรือ EXEs) ใน .NET ไมโครซอฟท์ ได้เตรียมพร้อมกับความสับสนนี้โดยแนะนำนิยามใหม่ที่เรียกว่า assembly ซึ่ง คือ ซอฟต์แวร์ คอมโพเนนต์ ที่สนับสนุน plug-and-play คล้าย ฮาร์ดแวร์ คอมโพเนนต์ .NET assembly นั้นมีความหมายเหมือนกับ COM โมดูล ในทางปฏิบัติ assembly สามารถบรรจุ หรือ อ้างถึง จำนวนของ type หรือ ไฟล์ทางกายภาพ (รวมทั้ง bitmap file, .NET PE files) ที่จำเป็นขณะรันไทม์ สำหรับการเอกซิกิว ให้สมบูรณ์ นอกจากนี้ยังเป็นที่อยู่ของ IL โค้ด assembly นั้น เป็น หน่วยพื้นฐาน สำหรับการกำหนด version การ deployment การบริหารความปลอดภัย การเอกซิกิวแบบ side-by-side การแชร์ การนำกลับมาใช้ใหม่

### 6.2.1 Unique Identities

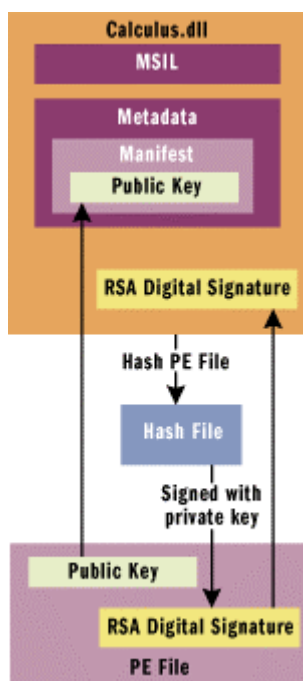
type ที่มีความเป็นเอกลักษณ์ มีความสำคัญ ใน COM และ .NET การให้จำนวนหลากหลายของ GUIDs ใน COM การพัฒนา และการดีพลอย นำเปื้อเพราะ จะต้องใช้ตัวเลขเหล่านี้ในโค้ด ใน .NET จะต้องอ้างอิงถึง type ที่เจาะจงโดยมันสามารถอ่านชื่อและ namespaces ของมันได้ เพราะว่า ชื่อที่อ่านได้ของมัน และ namespaces ไม่สามารถที่จะเป็นเอกลักษณ์ได้ .NET จึงใช้ unique public/private key pairs ทำให้ assembly ทั้งหมดนั้นสามารถแชร์ได้ โดยที่แอปพลิเคชันทั้งหลายต้องสร้างด้วย public/private key pair โดย assembly creator สามารถ sign assembly ด้วย private key และ ใครก็ตามสามารถ ตรวจสอบ digital signature นั้น โดยใช้ public key ของ assembly creator

ในการ sign assembly นั้น คุณจะต้องใช้ public/private key pair ขณะการสร้าง assembly คอมไพเลอร์สร้าง hash ของ assembly files แล้ว sign hash นั้นด้วย private key และ เก็บผลของ digital signature ในส่วนที่จองไว้ของ PE ไฟล์ public key ก็จะทำการเก็บไว้ใน assembly ด้วย

ในการตรวจสอบ digital signature ของ assembly CLR จะใช้ public key ของ assembly ในการถอดรหัส digital signature โดย CLR จะใช้ข้อมูลใน manifest ของ assembly ในการสร้าง hash แบบเปลี่ยนแปลงตลอดเวลา โดยค่าของ hash นี้จะเปรียบเทียบกับค่าของ hash ดั้งเดิม ค่าเหล่านี้จะต้องตรงกัน

เมื่อคุณหรือใครบางคน สร้าง แอปพลิเคชันขึ้นมาต้องการใช้แชร์ assembly assembly manifest ของ แอปพลิเคชันจะต้องรวม 8-byte hash ของ public key ของ แชร์ assembly ด้วย เมื่อทำการรันแอปพลิเคชัน

เคชัน CLR ก็จะอนุพันธ์ 8-byte hash จาก public key ของแชร assembly และทำการเปรียบค่านี้นี้ด้วย ค่า hash ที่ทำการเก็บไว้ใน assembly manifest ของคุณถ้าตรงกัน CLR ก็จะโหลด assembly ที่ถูกต้องสำหรับคุณ



รูปที่ 6-1 การทำ Digital Signature

## Interfaces

Interface สันนิษฐานคอนเซปต์เดียวกับ C++ abstract base class (ABC) ด้วย pure virtual function เท่านั้น อย่างแท้จริง ABC คือ คลาสที่ประกาศเพียงหนึ่งหรือมากกว่านั้น ของ pure virtual function คุณต้องกำหนด แต่ไม่ต้อง อิมพลีเมนต์ คลาสที่อนุพันธ์จากอินเทอร์เฟซ ของคุณต้องทำการอิมพลีเมนต์อินเทอร์เฟซของคุณ โดยอินเทอร์เฟซจะบรรจุ เมธอด คุณสมบัติ indexers และ อีเวนต์ ใน .NET คลาสสามารถอนุพันธ์จาก หลายอินเทอร์เฟซได้

### 6.2.2 Working .NET Component

ตัวอย่างการเขียน Assembly ในภาษา c++

```
#using <mscorlib.dll>

using namespace System;
```

```

namespace Lang
{
    __gc __interface ISteering
    {
        void TurnLeft( );
        void TurnRight( );
    };

    __gc class Vehicle : public ISteering
    {
        public:
            void TurnLeft( )
            {
                Console::WriteLine("Vehicle turns left.");
            }
            void TurnRight( )
            {
                Console::WriteLine("Vehicle turns right.");
            }
            virtual void ApplyBrakes( ) = 0;
    };

    public __gc class Car : public Vehicle
    {
        public:
            void ApplyBrakes( )
            {
                Console::WriteLine("Car trying to stop.");
                throw new Exception("Brake failure!");
            }
    };
} // This brace ends the Lang namespace.

```

## ตัวอย่างการเขียน Assembly ในภาษา VB.NET

Imports System

Namespace Lang

Interface ISteering

Sub TurnLeft()

Sub TurnRight()

End Interface

MustInherit Class Vehicle

Implements ISteering

Public Sub TurnLeft() Implements ISteering.TurnLeft

Console.WriteLine("Vehicle turns left.")

End Sub

Public Sub TurnRight() Implements ISteering.TurnRight

Console.WriteLine("Vehicle turn right.")

End Sub

Public MustOverride Sub ApplyBrakes()

End Class

Class Car

Inherits Vehicle

Public Overrides Sub ApplyBrakes()

Console.WriteLine("Car trying to stop.")

throw new Exception("Brake failure!")

End Sub

End Class

End Namespace

ตัวอย่างการเขียน Client Application เรียกใช้ ซึ่งต้องทำการ add DLL reference ใน tool  
VS.NET

```
using System;
namespace ConsoleApplication2
{
    class Drive
    {
        [STAThread]
        static void Main(string[] args)
        {
            Lang.Car bank = new Lang.Car();
            bank.ApplyBrakes();
        }
    }
}
```

### 6.2.3 Deployment

จากที่เราได้เห็นตัวอย่างในหัวข้อที่ผ่านมา .NET Assembly ทุกตัว เป็น binary component ซึ่งคุณสามารถ นำไป plug กับ component หรือ application อื่นๆได้ โดยที่ไม่ต้องการ source code และไม่ต้องการ plumbing code ด้วย

ใน .NET คอมโพเนนต์ สามารถเป็นได้ 2 แบบ ทั้ง private ที่ไม่ประกาศ(unpublished) และถูกใช้เพียงโคลเอนต์บางตัวเท่านั้น

#### 6.2.3.1 Private Components

private component ถูกใช้กับ โคลเอนต์ที่เจาะจงเท่านั้น ซึ่งมีสองวิธีในการทำ คือ

**1. One-directory deployment** ใช้ คอมโพเนนต์ เหล่านี้จากไดเรกทอรีเดียวกันกับ โคลเอนต์

ถ้าเราทำการลบหรือย้ายไฟล์คอมโพเนนต์ออกไป เมื่อเรารันโปรแกรม จะทำให้เกิด exception ออกมา

**2. Multiple-directory deployment** ใช้คอมโพเนนต์จากไคลเอนต์ที่ไคลเอนต์สามารถเข้าถึงได้ การทำแบบนี้ขั้นตอนสร้างต้องทำการอ้างอิง path ของ dll ที่อ้างอิงด้วย



รูปที่ 6-2 Multiple-directory tree of component

```
csc /r:..\vehicle\vehicle.dll /t:library /out:plane.dll plane.cs
```

ในขณะทำการอ้างอิงคอมโพเนนต์เหล่านี้ตอน build time CLR จะไม่ทำการเช็ค เวอร์ชัน ของ private คอมโพเนนต์

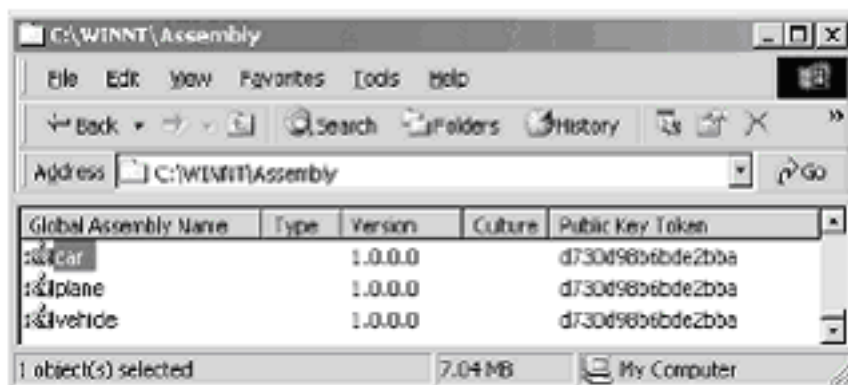
การ Install เพียงทำการ xcopy คือ ก๊อปปี้ไฟล์จากไคลเอนต์ของต้นฉบับ ไปยัง ไคลเอนต์ที่ต้องการ

การ Remove เพียงทำการลบไฟล์(หรือไคลเอนต์)เหล่านั้นทิ้งไปก็พอ เราไม่ต้องกังวลเกี่ยวกับการเขียนโค้ดเพื่อทำการเก็บข้อมูลต่างๆลงไปใน registry และเรื่องอื่นๆเกี่ยวกับการ registry เลย

### 6.2.3.2 Shared Components

shared Components เรียกอีกอย่างว่า system component ซึ่งสามารถถูกเรียกใช้ได้จาก โปรแกรมไคลเอนต์ใดๆก็ได้ แต่ว่า เราต้องทำการประกาศหรือทำการ ลงทะเบียน(register) ลงใน system Global Assembly Cache (GAC) เมื่อเราทำการลงทะเบียนเสร็จ มันก็เป็นเหมือน system dll ซึ่ง process ใดๆในระบบสามารถเรียกใช้ได้

ข้อสำคัญในการลงทะเบียนใน GAC คือ คอมโพเนนต์ต้องทำเกี่ยวกับ originator และ version ด้วย และด้วยสองส่วนนี้ ทำให้เราสามารถสร้าง คอมโพเนนต์เดียวกันในหลายๆเวอร์ชันและรันบนเครื่องเดียวกันได้ โดยไม่มีปัญหา



รูปที่ 6-3 แชร่คอมโพเนนต์ที่เก็บใน GAC

#### 6.2.4 ขั้นตอนการลงทะเบียนคอมโพเนนต์

1. ใช้ share named utility (sn.exe) ในการสร้าง random public/private key และบันทึกเก็บไว้ในไฟล์ เช่น originator.key

sn -k originator.key

2. สร้าง assembly(คอมโพเนนต์) พร้อมด้วย assembly version number และ key information จาก originator.key

Imports System

Imports System.Reflection

<Assembly:AssemblyVersion("1.0.0.0")>

<assembly:AssemblyKeyFile("../key\\originator.key")>

Public Class Car

Inherits Vehicle

Overrides Public Sub TurnLeft( )

Console.WriteLine("Car turns left.")

End Sub

Overrides Public Sub TurnRight( )

Console.WriteLine("Car turns right.")

End Sub



```

Overrides Public Sub ApplyBrakes( )
    Console.WriteLine("Car trying to stop.")
    Console.WriteLine("ORIGINAL VERSION - 1.0.0.0.")
    throw new Exception("Brake failure!")
End Sub
End Class

```

3. ใช้ .NET Global Assembly Cache Utility(gacutil.exe) ในการลงทะเบียน assembly ลงใน GAC ก็เป็นอันเสร็จเป็น shared component สามารถเรียกใช้จาก client ใดๆก็ได้

gacutil /i car.dll

การ uninstall shared component เหล่านี้ ก็เพียงแค่ทำการ ลบไฟล์ assembly ที่เหมาะสมก็เสร็จ มันก็จะออกจาก GAC

### 6.2.5 Building and testing the drive.exe

เราสร้างแอปพลิเคชัน drive.exe เราต้องมีการ include คอมโพเนนต์ที่ต้องการเรียกใช้เข้าไปด้วย เราเรียกตอนนี้ว่า build time ซึ่งเมื่อสร้างแล้ว ได้ผลลัพธ์ไฟล์ drive.exe

ORIGINAL VERSION - 1.0.0.0.

Adding new versions

ในการใช้ shared assembly นั้นจะมีการใช้ประโยชน์เกี่ยวกับเรื่อง version policies ในขณะที่โหลด shared assemblies ทั้งหมด ซึ่ง CLR จะทำการโหลด assembly version ที่เราใช้ในขณะ build time แต่เราก็สามารถที่จะสั่งให้ CLR โหลด เวอร์ชันที่ต้องการได้ โดยสร้าง application configuration ขึ้นมาเพื่อบอกให้เรียกใช้เวอร์ชันที่เราต้องการ

แก้ไข code ใหม่เป็น

```
<Assembly:AssemblyVersion("1.0.1.0")>
```

```
Overrides Public Sub ApplyBrakes( )
```

```
    Console.WriteLine("Car trying to stop.")
```

```
    Console.WriteLine("BUILD NUMBER change - 1.0.1.0.")
```

```
End Sub
```

เมื่อเราคอมไพล์ออกมาจะได้ไฟล์ที่อาจจะมีชื่อซ้ำกันได้ แต่ยังสามารถใช้ได้เนื่องจากเวอร์ชันไม่เหมือนกัน ถ้าเราทำการรัน โปรแกรม drive.exe ก็จะได้ผลลัพธ์ คือ

ORIGINAL VERSION - 1.0.0.0.

ถ้าต้องการให้โหลด assembly เวอร์ชันใหม่ เราต้องทำการสร้าง config ให้มัน ในชื่อของ drive.exe.config ไว้ในไดเรกทอรีเดียวกันกับ drive.exe

```
<?xml version="1.0"?>
<configuration>
  <runtime>
    <assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
      <dependentAssembly>
        <assemblyIdentity name="car"
          publicKeyToken="D730D98B6BDE2BBA"
          culture="" />
        <bindingRedirect oldVersion="1.0.0.0"
          newVersion="1.0.1.0" />
      </dependentAssembly>
    </assemblyBinding>
  </runtime>
</configuration>
```

เมื่อเราทำการรัน drive.exe อีกครั้ง ผลลัพธ์ที่ได้จะเปลี่ยนไป จะไปเรียกใช้ dll เวอร์ชัน 1.0.1.0 แทน BUILD NUMBER change - 1.0.1.0.

มีส่วนที่ควรรู้สองส่วนในเรื่องของ config คือ 1.ชื่อของ assemblyIdentity tag จะเป็น ชื่อของ assembly ที่ใช้สำหรับคนอ่าน (human-readable) ซึ่งเก็บใน GAC และ 2.publicKeyToken attribute จะเก็บค่าของ public key ซึ่งเป็นเลข 8 byte hash ของ public key ในการสร้าง(build) คอมไพล์

ในการดูค่า 8-byte hash นี้ สามารถดูได้จากหลายวิธี เช่นจาก Shell Cache Viewer , IL dump of component หรือใช้ sn.exe  
sn -T car.dll

### 6.3 Enterprise Services (COM+) in .NET

.NET สามารถใช้งานทุกบริการของ COM+ ที่มีให้ใช้งานได้ เช่น

- 1.Transaction
- 2.Just-In-Time (JIT)
- 3.Object Pooling
- 4.Security

#### 6.3.1 Transaction

ในการเขียน managed แอปพลิเคชันนั้นในการใช้บริการ คลาสต่างๆต้องทำการ derive จาก ServicedComponent ระบุแอตทริบิวต์ ต่างๆเพื่อระบุ บริการที่ต้องการจริงๆ

```
BankComponentClient
using system;
using BankComponent'
namespace BankComponentClient
{
    class Client
    {
        public static int Main()
        {
            Account act = new Account ();
            act.Post (5,100);
            act.Dispose ();
            return 0;
        }
    }
}
```

## BankComponent Server

```

using System.EnterpriseServices

[assembly: ApplicationName ("BankComponent")]
[assembly: AssemblyKeyFileAttribute("Demos.snk")]

namespace BankComponentServer
{
    [Transaction(TransactionOption.Required)]
    public class Account : ServicedComponent
    {
        [AutoComplete]
        public bool Post (int accountNum, double amount)
        {
            // Updates the database, no need to call SetComplete.
            //Calls SetComplete automatically if no exception is thrown.
        }
    }
}

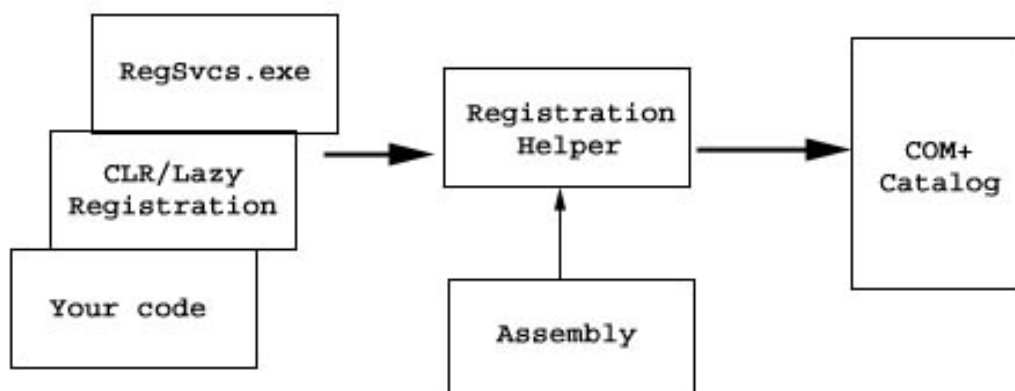
```

BankComponent Server code แสดงการใช้งานของ Account คลาสใน .NET ซึ่งใช้ การ ทรานแซกชัน โดยคลาส Account จะ derive จากคลาส System.Enterprise.ServicedComponent ค่าแอททริบิวต์ของการทรานแซกชัน ก็จะถูกกำหนด ให้เป็นต้องการ การซิงโครไนซ์ และ JIT ก็จะถูกกำหนดให้ใช้งานโดยอัตโนมัติเพราะได้กำหนดค่าแอททริบิวต์ของทรานแซกชันแล้ว

จากโค้ดจะเห็นว่าใน BankComponent Server namespace แสดงให้เห็นว่าเป็นการง่ายในการที่จะใช้ บริการของ COM+ ใน .NET

### 6.3.2 Lazy Registration

เมื่อไหร่ก็ตามไคลเอนต์แอปพลิเคชันทำการสร้างอินสแตนซ์ของคลาสที่ derive มาจาก ServicedComponent runtime จะทำการค้นหาว่าได้ทำการรีจิสเตอร์ assembly เรียบร้อยหรือยังถ้าไม่พบ เซอร์วิสคอมโพเนนต์ทั้งหมดใน assembly ก็จะถูกรีจิสเตอร์ลงใน COM+ แอปพลิเคชันซึ่งเรียกว่า lazy registration ซึ่ง lazy registration มีประโยชน์ในช่วงระหว่างการพัฒนา



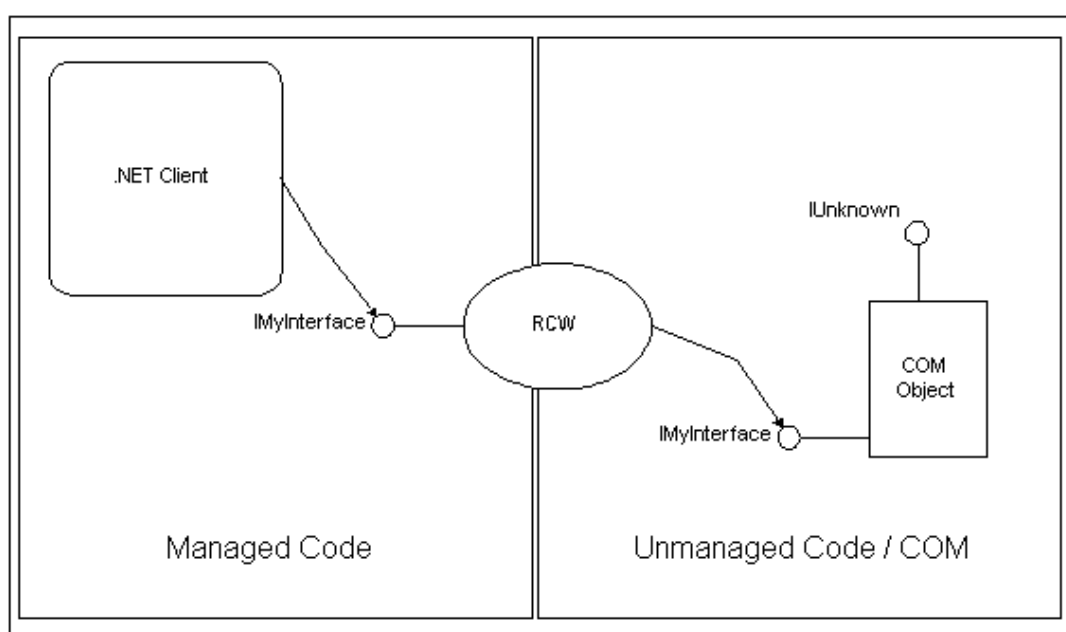
รูปที่ 6-4 แสดงการทำงานของ *Lazy Registration*

#### 6.4 .NET Interoperability

คือความสามารถที่ทำให้ .NET สามารถใช้งานร่วมกับ COM คอมโพเนนต์ แบบเดิมได้ดังนี้

##### 6.4.1 การเรียก COM คอมโพเนนต์จาก .NET

เมื่อ COM ออบเจ็กต์ถูกเรียกจาก .NET รันไทม์ก็จะทำการสร้าง Runtime Callable Wrapper (RCW) โดยที่ RCW ก็จะทำหน้าที่เหมือนพร็อกซีสำหรับ unmanage ออบเจ็กต์



รูปที่ 6-5 แสดงการเรียก *COM* คอมโพเนนต์จาก *.NET*

RCW จะรับผิดชอบหน้าที่ในการติดต่อระหว่าง .NET ไคลเอนต์ โค้ด และ COM คอมโพเนนต์รวมทั้ง

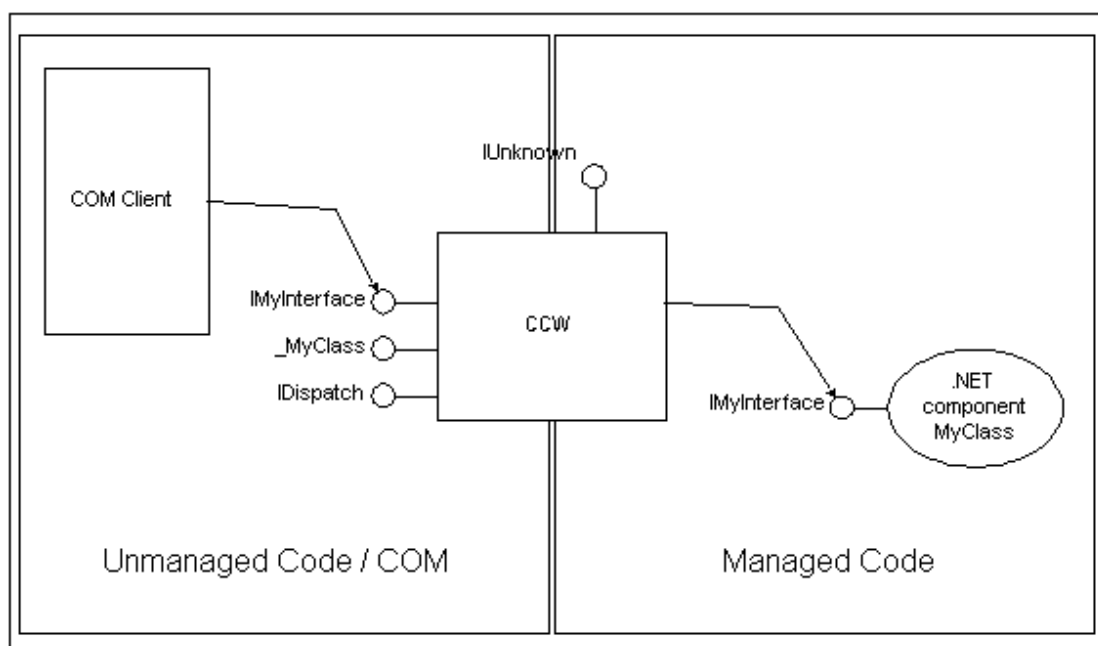
- สร้างและการ binding ที่มี COM คอมโพเนนต์ เป็นรากฐาน
- การใช้ อินเตอร์เฟซ และ องค์กรประกอบขอ Synthesizing อินเตอร์เฟซไปเป็น managed form
- ทำการแปลและทำการมาแซล ระหว่างสภาพแวดล้อม
- บริหาร ไลฟไทม์ ของ wrapped COM object
- ทำการแปลค่า COM HRESULT ไปเป็น .NET exception

RCW เป็น manage ออบเจกต์และอยู่ใน heap ซึ่งดูแลโดย CLR เพราะกับ manage ออบเจกต์อื่นๆก็จะอ้างไปที่ RCW ซึ่งจะถูกลบโดย รันไทม์ และ RCW ก็คือหัวข้อไปสู่ garbage collection

คือความสามารถที่ทำให้ .NET สามารถใช้งานร่วมกัน COM คอมโพเนนต์ แบบเดิมได้ดังนี้

#### 6.4.2 การเรียก .NET คอมโพเนนต์จาก COM

เมื่อ .NET คอมโพเนนต์ถูกเรียกจาก COM รันไทม์ก็จะสร้าง wrapper ออบเจกต์เพื่อเชื่อมต่อช่องว่างระหว่างสภาพแวดล้อมซึ่งในกรณีนี้รันไทม์จะสร้าง COM callable wrapper (CCW) รันไทม์ก็จะอ่านรายละเอียด type สำหรับคอมโพเนนต์จาก assembly metadata และสร้าง CCW ที่เข้ากันได้ คล้ายกับ RCW แต่ CCWจะทำตัวเป็นพร็อกซี unmanaged COM code กับ managed .NET code



รูปที่ 6-6 แสดงการเรียก .NET คอมโพเนนต์จาก COM

CCW รับผิดชอบทั้งหมดระหว่าง COM ไคลเอนต์กับ managed อ็อบเจกต์รวมทั้ง

- สร้างและมีการ binding ที่มี managed อ็อบเจกต์เป็นรากฐาน
- ทำการสังเคราะห์อินเทอร์เฟซที่สำคัญหลายตัวของ COM (เช่น IUnknown และ Idispach) ขึ้น

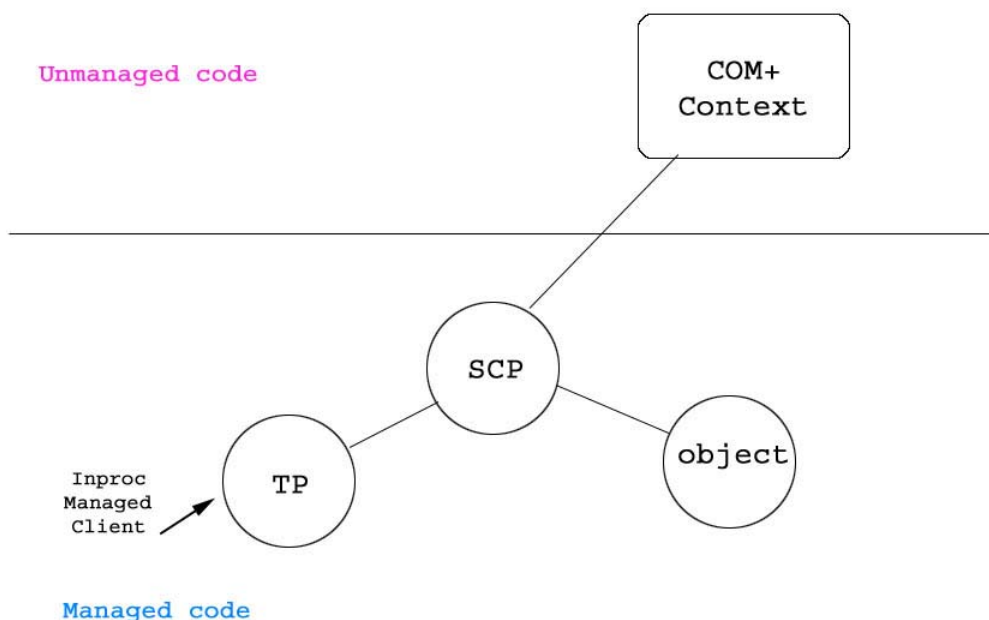
กับข้อมูล type ของอ็อบเจกต์

- ทำการมาเชลและทำการแปลระหว่างสภาพแวดล้อม
- บริหารไลฟ์ไทม์ของ .NET คอมโพเนนต์
- แปล .NET exceptions ไปเป็นค่า COM HRESULT

CCW ก็คือ unmanaged COM อ็อบเจกต์ซึ่งอยู่ที่ heap และ เป็น reference-counted COM อ็อบเจกต์เดิม CCW ไม่ใช่ garbage collected แต่เป็นการทำลาย โดยปล่อย ไคลเอนต์ที่ทำการอ้างถึงล่าสุด เมื่อ CCW ถูกทำลายแล้วก็จะทำการ wrap ก็จะถูกรับ mark สำหรับ garbage collection

#### - Activation

Enterprise Services สร้างจากคอนเซปต์ ของ คอนเท็กซ์ ซึ่ง คอนเท็กซ์คือสภาพแวดล้อมสำหรับ อ็อบเจกต์ที่มีความต้องการสำหรับการเอ็กซิคิวท์ที่คล้ายคลึงกัน เซอร์วิสสามารถถูกบังคับให้ปฏิบัติตาม ระหว่าง activation และ/หรือ เมธอด เรียกว่า interception ถึงแม้ว่าบริการของ COM+ จะถูกเขียนเป็น unmanaged code การผนวก COM+ services กับ .NET ลึกเกินกว่าแค่การใช้ COM interop technology in .NET ที่ปราศจากการ derive จาก ServicedComponent การ registration โปรเซสซึ่งจะไม่เกิดผลตามที่ปรารถนา

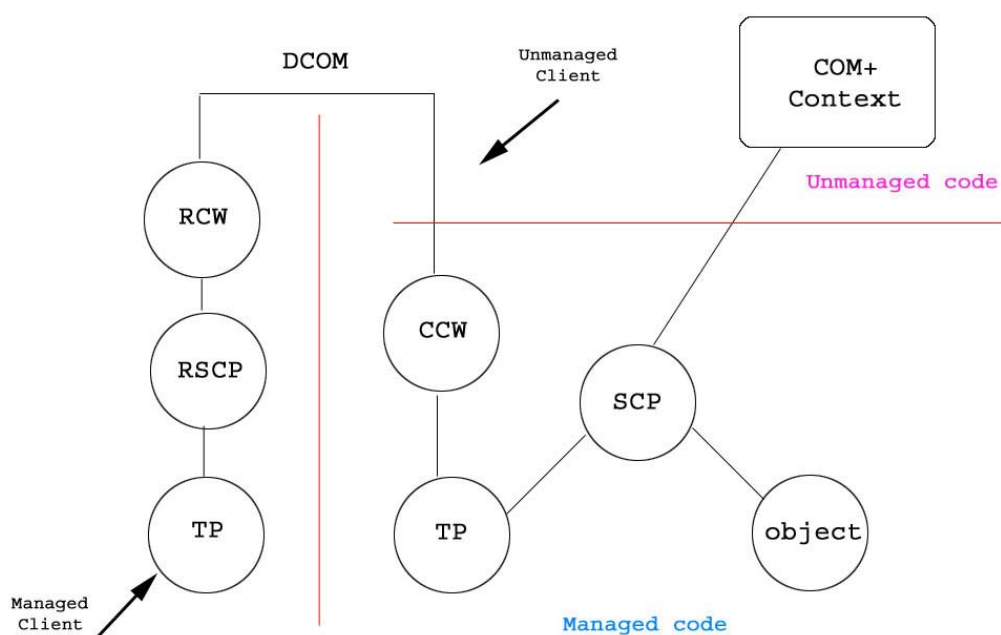


รูปที่ 6-7 in-process activation

activation มี 3 กรณีดังต่อไปนี้แบบแรก in-process activation คือมีการข้าม context boundary แบบที่สอง cross-domain เป็นการข้ามทั้ง context และ app domain แบบที่สาม cross-process ข้ามเครื่อง/ข้ามprocess และข้าม context boundaries

ในกรณี in-process activation จากรูปแสดงให้เห็นว่า reference เป็นแบบunmarshaled ซึ่งเป็น direct reference ไปที่ transparent proxy (TP) กรณี cross app-domain reference เป็นแบบ unmarshaled ซึ่งคือ .NET remoting proxy

ในกรณี Cross process หรือ Cross machine reference ต้องการมากกว่าการ unmarshaled โดย COM interop ทำการเรียก IManagedObject ถูกอิมพลีเมนต์โดย ServicedComponent ระหว่าง activation และการมาแชล remote serviced component proxy (RSCP) ทำการเรียก IServicedComponentInfo ระหว่าง activation



รูปที่ 6-8 cross-process activation

#### - Contexts

คลาส ContextUtil ใช้ในการเข้าถึง COM+ object context ที่เกี่ยวข้อง และ คุณสมบัติต่างๆของมัน



### 6.5 .NET Component VS COM

|              | .NET Component                                                                                                                              | COM                                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Architecture | Object oriented to the core of language                                                                                                     | Just the wrapper of the binary component                                                                    |
| Deployment   | <ul style="list-style-type: none"> <li>- Just copy the file</li> <li>- Varied versions of same component can exist in the system</li> </ul> | <ul style="list-style-type: none"> <li>- Require registration</li> <li>- Incompatibility problem</li> </ul> |
| Language     | All language are 1 <sup>st</sup> class                                                                                                      | Only c++ can be low –level COM programming                                                                  |
| Code         | Eliminate plumbing code                                                                                                                     | Plumbing code to support IUnknow, class factory , component lifetime                                        |

**ตาราง 6-1 ตารางเปรียบเทียบ ระหว่าง .NET Component กับ COM**